In The Name of God

Ali AsharDegarsi 1404122615101 HW#2 DIP

1. Write a MATLAB or Python program to read a grayscale image and display its size, data type, and intensity range.

```python
import cv2
import numpy as np


def analyze_image(file_path):
    # Read the image
    # The second argument, 0, or cv2.IMREAD_GRAYSCALE,
converts the image to grayscale upon loading
    img = cv2.imread(file_path, 0)

    # Check if the image was loaded successfully
    if img is None:
        print("Error: Unable to load image. Please check the
file path.")
        return

    print("--- Image Properties ---")

    # 1. Display Size
    # The shape attribute returns a tuple (height, width)
for a grayscale image
    height, width = img.shape
    print(f"Size (Height x Width): {height} x {width}")

    # 2. Display Data Type
    print(f"Data Type: {img.dtype}")

    # 3. Display Intensity Range
    # np.min and np.max find the darkest and lightest pixel
values
    min_val = np.min(img)
    max_val = np.max(img)
    print(f"Intensity Range: {min_val} to {max_val}")
```

```
if __name__ == "__main__":
    image_path = 'grayscale_image.jpg'

    analyze_image(image_path)
```

```
--- Image Properties ---
Size (Height x Width): 1000 x 667
Data Type: uint8
Intensity Range: 1 to 255
```

---

2. Write a program to convert an RGB image to grayscale.

```
import cv2

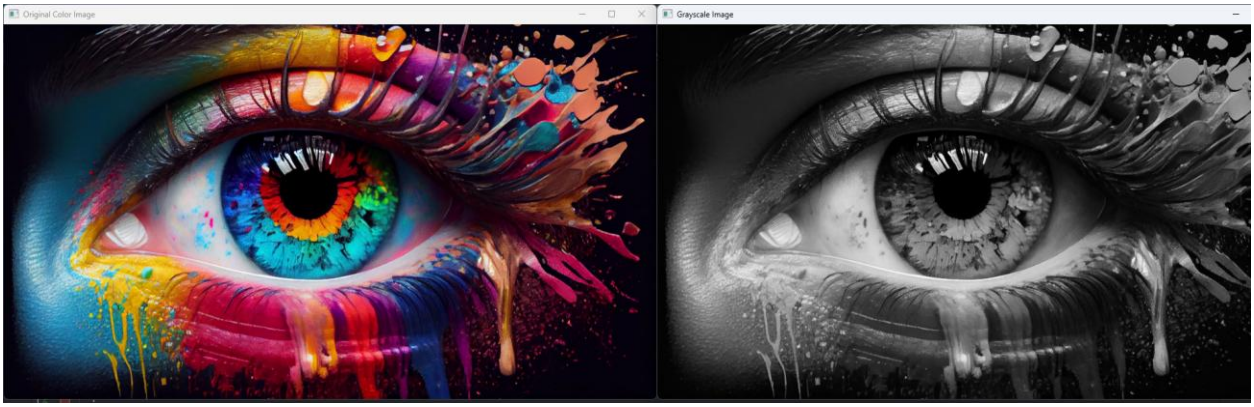
# 1. Read the image
# OpenCV reads images in BGR format by default, not RGB
input_path = 'color_image.jpg'
image = cv2.imread(input_path)

# Check if image loaded successfully
if image is None:
    print("Error: Could not load image.")
else:
    # 2. Convert BGR to Grayscale
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 3. Display the results
    cv2.imshow('Original Color Image', image)
    cv2.imshow('Grayscale Image', gray_image)

    # 4. Save the grayscale image
    cv2.imwrite('grayscale_output.jpg', gray_image)

    # Wait for a key press and close windows
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



3. Reduce an 8-bit grayscale image to 4-bit quantization (16 levels).

```
import cv2
import numpy as np


def quantize_image(image_path):
    # 1. Read the image in Grayscale (8-bit)
    img = cv2.imread(image_path, 0)

    if img is None:
        print("Error: Could not load image. Check path.")
        return

    # 2. Perform Quantization (Reduce to 4-bit)
    # Bit shifting right by 4 divides the value by 16.
    # Values 0-255 become values 0-15.
    quantized_raw = img >> 4

    # 3. Scale back to 0-255 range for Visualization
    # back to the full intensity range (0-255).
    # Factor = 255 / 15 = 17
    quantized_display = (quantized_raw *
17).astype(np.uint8)

    # 4. Display the results
    cv2.imshow('Original (8-bit)', img)
    cv2.imshow('Quantized 4-bit (16 Levels)',
quantized_display)

    print(f"Original Range: 0 to 255")
```

```
    print(f"Quantized Raw Values: {np.min(quantized_raw)} to
{np.max(quantized_raw)}")
    print(f"Display Scaled Values:
{np.min(quantized_display)} to
{np.max(quantized_display)}")

    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":
    image_file = 'grayscale_image.jpg'
    quantize_image(image_file)
```



---

4. Write a program to downsample an image by a factor of 2.

```
import cv2


def downsample_numpy(image_path):
    # 1. Read the image
    img = cv2.imread(image_path)

    if img is None:
```

```
        print("Error: Could not load image.")
        return

    # 2. Downsample by slicing
    # Syntax: array[start:stop:step]
    # [::2] means take every 2nd pixel starting from 0
    downsampled_img = img[::2, ::2, :]

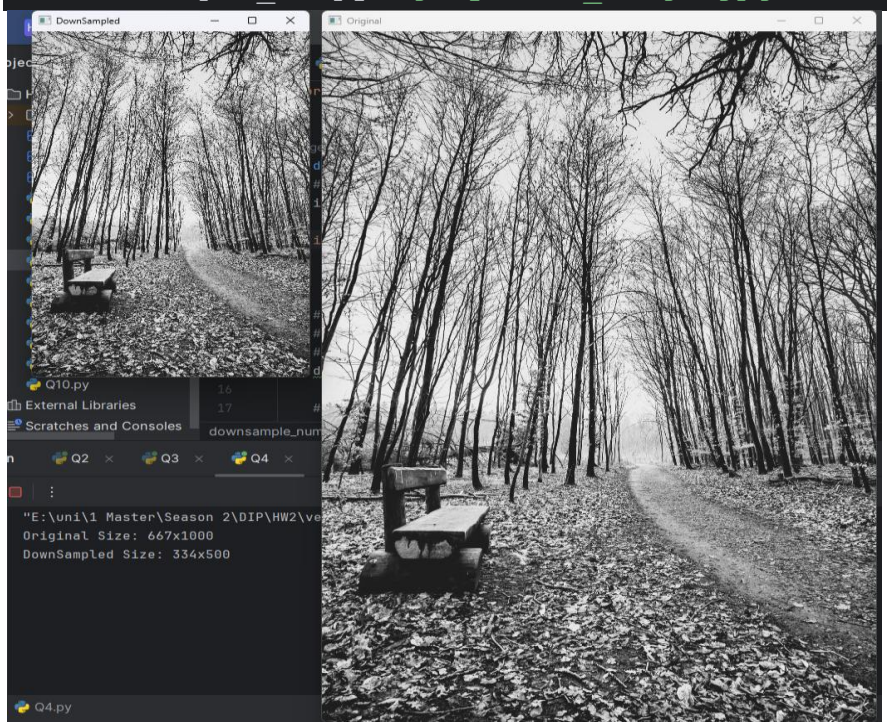
    # 3. Display Info and Images
    h, w = img.shape[:2]
    nh, nw = downsampled_img.shape[:2]

    print(f"Original Size: {w}x{h}")
    print(f"DownSampled Size: {nw}x{nh}")

    cv2.imshow('Original', img)
    cv2.imshow('DownSampled', downsampled_img)

    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":
    downsample_numpy('grayscale_image.jpg')
```



5. Upsample an image by a factor of 2 using interpolation.

```
import cv2


def upsample_image(image_path):
    # 1. Read the image
    img = cv2.imread(image_path)

    if img is None:
        print("Error: Could not load image.")
        return

    # 2. Upsample by a factor of 2
    # We use None for the dsize (destination size) because
we are defining the scale factors instead.
    # fx and fy are the scale factors along the horizontal
and vertical axis.
    # interpolation=cv2.INTER_CUBIC is generally best for
upsampling as it produces smoother results.
    upsampled_img = cv2.resize(img, None, fx=2, fy=2,
interpolation=cv2.INTER_CUBIC)

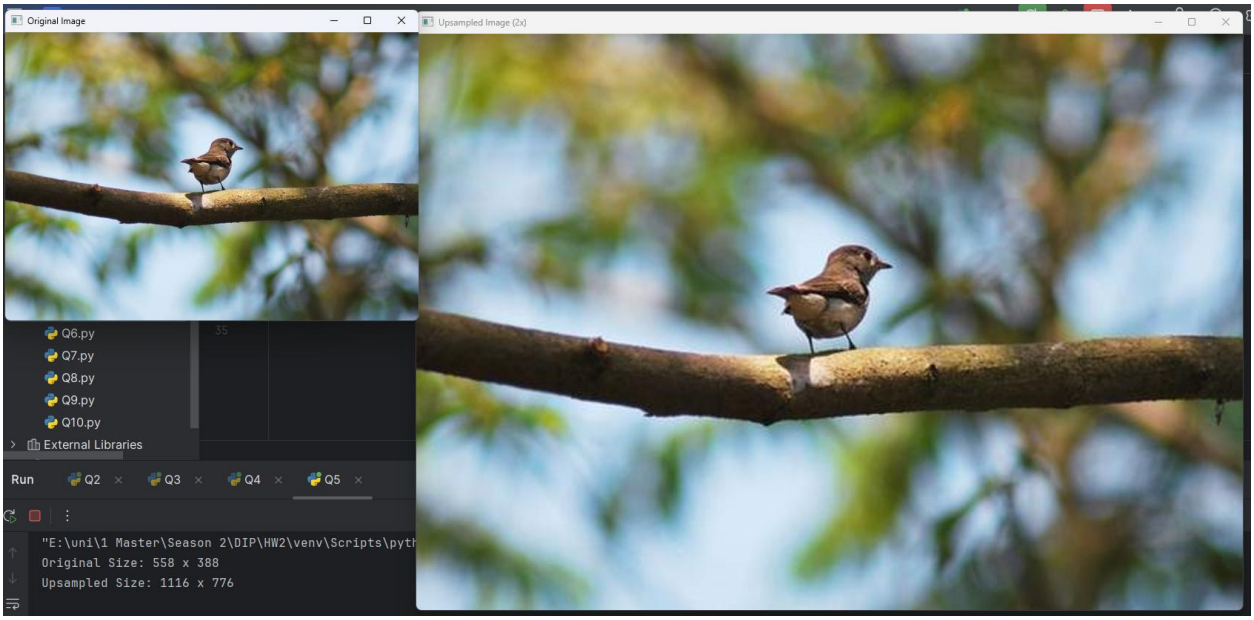
    # 3. Display dimensions
    h, w = img.shape[:2]
    uh, uw = upsampled_img.shape[:2]

    print(f"Original Size: {w} x {h}")
    print(f"Upsampled Size: {uw} x {uh}")

    # 4. Display the results
    cv2.imshow('Original Image', img)
    cv2.imshow('Upsampled Image (2x)', upsampled_img)

    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":
    upsample_image('grayscale_image.jpg')
```



---

6. Write a program to generate the negative of a grayscale image.

```python
import cv2


def generate_negative(image_path):
    # 1. Read the image in Grayscale mode (0)
    original_img = cv2.imread(image_path, 0)

    if original_img is None:
        print("Error: Could not load image. Check the path.")
        return

    # 2. Generate the Negative
    # We subtract the image matrix from 255.
    # NumPy handles this operation element-wise for every pixel simultaneously.
    negative_img = 255 - original_img

    # 3. Display the results
    cv2.imshow('Original Grayscale', original_img)
    cv2.imshow('Negative Image', negative_img)

    # 4. Save the result
    cv2.imwrite('negative_output.jpg', negative_img)
```

```
    print("Negative image generated and saved as
'negative_output.jpg'")

    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":
    generate_negative('grayscale_image.jpg')
```



7. Write a program to extract and print the intensity value of a specific pixel (x,y).

```
import cv2


def get_pixel_intensity(image_path, x, y):
    # 1. Read the image in Grayscale mode
    img = cv2.imread(image_path, 0)
```

```
    if img is None:
        print("Error: Could not load image.")
        return

    # 2. Get image dimensions (Height = Rows, Width =
Columns)
    height, width = img.shape

    # 3. Check if coordinates are within the image bounds
    if x < 0 or x >= width or y < 0 or y >= height:
        print(f"Error: Coordinates ({x}, {y}) are out of
bounds.")
        print(f"Valid range: x (0 to {width - 1}), y (0 to
{height - 1})")
        return

    # 4. Extract the Intensity
    # Access the pixel at row 'y' and column 'x'
    intensity_value = img[y, x]

    # 5. Print the result
    print(f"--- Pixel Info ---")
    print(f"Coordinates (x, y): ({x}, {y})")
    print(f"Intensity Value:    {intensity_value}")


if __name__ == "__main__":
    # Replace with your image path
    image_file = 'grayscale_image.jpg'

    # Define the coordinates you want to check
    target_x = 100
    target_y = 50

    get_pixel_intensity(image_file, target_x, target_y)
```

```
--- Pixel Info ---
Coordinates (x, y): (100, 50)
Intensity Value:    87
```

8. Extract the most significant bit (MSB) of an image.

```
import cv2


def extract_msb_plane(image_path):
    # 1. Read the image in Grayscale (8-bit)
    img = cv2.imread(image_path, 0)

    if img is None:
        print("Error: Could not load image.")
        return

    # 2. Extract the MSB
    # Bitwise right shift by 7 positions.
    msb_binary = img >> 7

    # 3. Scale for Visualization
    # The values are currently 0 or 1, which would be
invisible on a screen.
    # Multiply by 255 to map 0 -> 0 (Black) and 1 -> 255
(White).
    msb_visual = msb_binary * 255

    # 4. Display the results
    cv2.imshow('Original Image', img)
    cv2.imshow('MSB Plane (8th Bit)',
msb_visual.astype('uint8'))

    print("Displaying Original Image and MSB Plane...")
    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":

    extract_msb_plane('grayscale_image.jpg')
```



9. Write a program to compute and display the histogram of an image.

```
import cv2
import matplotlib

matplotlib.use('TkAgg')

import matplotlib.pyplot as plt


def plot_grayscale_histogram(image_path):
    # 1. Read the image in Grayscale
    img = cv2.imread(image_path, 0)

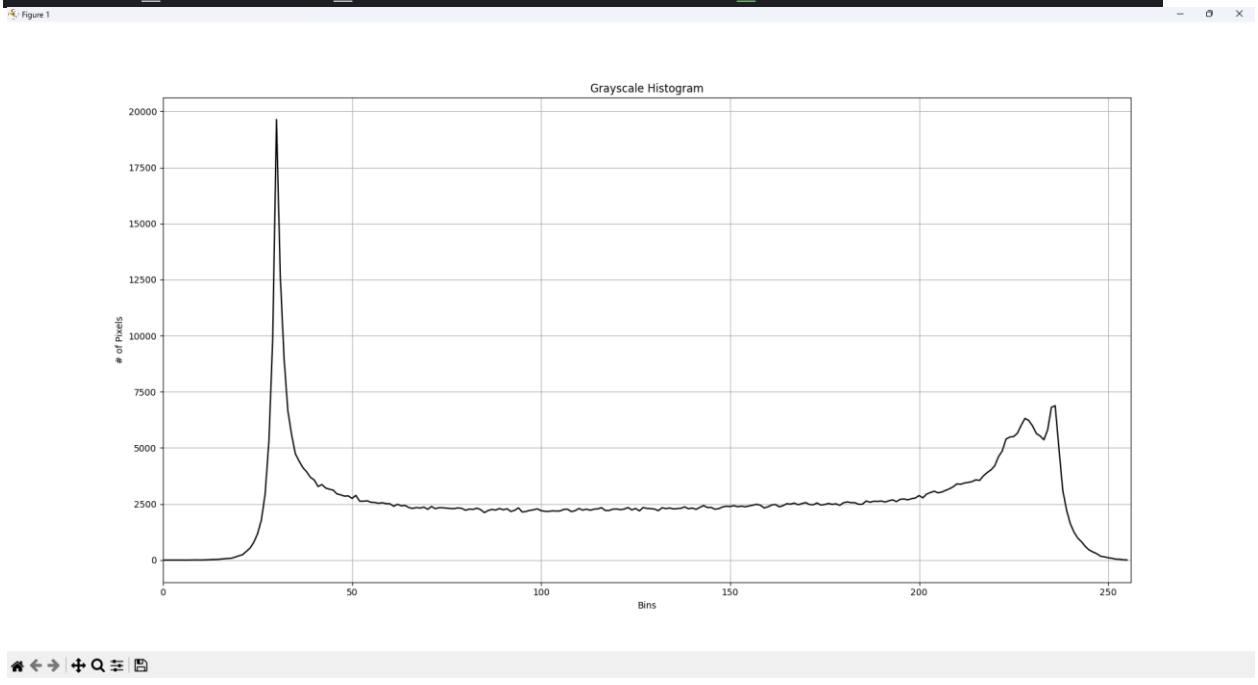
    if img is None:
        print("Error: Could not load image.")
        return
```

```
    # 2. Calculate the histogram
    hist = cv2.calcHist([img], [0], None, [256], [0, 256])

    # 3. Plot the histogram
    plt.figure()
    plt.title("Grayscale Histogram")
    plt.xlabel("Bins")
    plt.ylabel("# of Pixels")
    plt.plot(hist, color='black')
    plt.xlim([0, 256])
    plt.grid(True)

    plt.show()


if __name__ == "__main__":
    plot_grayscale_histogram('grayscale_image.jpg')
```



10. Write a program to crop a region of size 200×200 from the center of an image.

```
import cv2


def crop_center_200x200(image_path):
    # 1. Read the image
    img = cv2.imread(image_path)
```

```
    if img is None:
        print("Error: Could not load image.")
        return

    # 2. Get image dimensions (Height, Width)
    h, w = img.shape[:2]

    # 3. Define the crop size
    crop_size = 200

    # 4. Error Handling: Check if image is large enough
    if w < crop_size or h < crop_size:
        print(f"Error: Image is too small ({w}x{h}) to crop
a {crop_size}x{crop_size} region.")
        return

    # 5. Calculate start and end coordinates
    # Center coordinates
    center_x, center_y = w // 2, h // 2

    # Top-left corner (start)
    start_x = center_x - (crop_size // 2)
    start_y = center_y - (crop_size // 2)

    # Bottom-right corner (end)
    end_x = start_x + crop_size
    end_y = start_y + crop_size

    print(f"Cropping region from x:{start_x} to x:{end_x},
y:{start_y} to y:{end_y}")

    # 6. Crop using NumPy array slicing
    # Syntax: image[row_start:row_end, col_start:col_end]
    cropped_img = img[start_y:end_y, start_x:end_x]

    # 7. Display the results
    # Resize the window slightly just to make sure it's
visible
    cv2.imshow('Original Image', img)
    cv2.imshow('Cropped 200x200 (Center)', cropped_img)
```

```
    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":

    crop_center_200x200('image.jpg')
```

